Lecture 14

Kendra Burbank

Convergence of optimization algorithms

What converges?

When discussing convergence of optimization algorithms, it is important to make precise what exactly is converging - after all, this is what makes all of these a science. Given conditions on $\mathbf{x}_0$ and f, we can try to show that the sequence ($\mathbf{x}_k$) produced by our algorithm converges in the following senses:

1. Stationary point: $\lim_{k\to\infty} \mathbf{x}_k = \mathbf{x}_*$, where $\nabla f(\mathbf{x}_*) = \mathbf{0}$;
2. Candidate for local minimum: $\lim_{k\to\infty} \mathbf{x}_k = \mathbf{x}_*$, where $\nabla f(\mathbf{x}_*) = \mathbf{0}$ and $\nabla^2 f(\mathbf{x}_*) \succeq 0$;
3. Local minimum: $\lim_{k\to\infty} \mathbf{x}_k = \mathbf{x}_*$ where $\nabla f(\mathbf{x}_*) = \mathbf{0}$ and $\nabla^2 f(\mathbf{x}_*) \succ 0$.

These items are called **convergence**.

These are all too difficult to guarantee in general. Instead, we can prove weaker statements concerning the sequence of gradients and Hessians:

4. $\lim_{k\to\infty} \nabla f(\mathbf{x}_k) = \mathbf{0}$ (or $\liminf_{k\to\infty} \nabla f(\mathbf{x}_*) = \mathbf{0}$);
5. $\lim_{k\to\infty} \nabla f(\mathbf{x}_k) = \mathbf{0}$ and $\lim_{k\to\infty} \nabla^2 f(\mathbf{x}_k) \succeq 0$;
6. $\lim_{k\to\infty} \nabla f(\mathbf{x}_k) = \mathbf{0}$ and $\lim_{k\to\infty} \nabla^2 f(\mathbf{x}_k) \succ 0$.

These are called **functional convergence**.

For certain functions convergence and functional convergence are equivalent, but in general this is not true. Functional convergence is typically easier to establish.

Convergence of line search methods

We now return to the Wolfe conditions, and in particular, the convergence of line search methods with step size satisfying the Wolfe conditions.

Theorem

Suppose

1. $f \in C^1(\mathbb{R}^n)$,
2. $\nabla f \in \mathrm{Lip}_\gamma(\mathbb{R}^n)$ (*i.e.*, $\|\nabla f(\mathbf{x}) - \nabla f(\mathbf{y})\| \leq \gamma \|\mathbf{x} - \mathbf{y}\|$ *for all* $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$),
3. f *is bounded below,*

Theorem

4. $\mathbf{p}_k$ *and* α_k *satisfy Wolfe conditions:*

$$\begin{cases} f\left(\mathbf{x}_{k+1}\right) \leq f\left(\mathbf{x}_k\right) + c_1 \alpha_k \nabla f\left(\mathbf{x}_k\right)^\top \mathbf{p}_k \\ \nabla f\left(\mathbf{x}_{k+1}\right)^\top \mathbf{p}_k \geq c_2 \nabla f\left(\mathbf{x}_k\right)^\top \mathbf{p}_k, \end{cases}$$

5. *there exists* δ *such that* $\cos\theta_k = -\frac{\nabla f(\mathbf{x}_k)^\top \mathbf{p}_k}{\|\nabla f(\mathbf{x}_k)\|\|\mathbf{p}_k\|} \geq \delta > 0$ *for all* $k \in 0, 1, \ldots$.

Then $\lim_{k\to\infty} \nabla f\left(\mathbf{x}_k\right) = \mathbf{0}$.

Proof of Theorem 1.1

Subtracting $\nabla f(\mathbf{x}_k)^\top \mathbf{p}_k$ from both sides of the curvature inequality,

$$[\nabla f(\mathbf{x}_{k+1}) - \nabla f(\mathbf{x}_k)]^\top \mathbf{p}_k \geq (c_2 - 1) \nabla f(\mathbf{x}_k)^\top \mathbf{p}_k$$

By the Lipschitz condition and the Cauchy-Schwarz inequality,

$$[\nabla f(\mathbf{x}_{k+1}) - \nabla f(\mathbf{x}_k)]^\top \mathbf{p}_k \leq \|\nabla f(\mathbf{x}_{k+1}) - \nabla f(\mathbf{x}_k)\| \|\mathbf{p}_k\| \leq \alpha_k \gamma \|\mathbf{p}_k\|^2$$

Combining the two inequalities, we obtain

$$\alpha_k \gamma \left\| \mathbf{p}_k \right\|^2 \geq (c_2 - 1) \nabla f\left(\mathbf{x}_k\right)^\top \mathbf{p}_k, \quad \Rightarrow \quad \alpha_k \geq \frac{c_2 - 1}{\gamma} \frac{\nabla f\left(\mathbf{x}_k\right)^\top \mathbf{p}_k}{\left\| \mathbf{p}_k \right\|^2}$$

Substituting this inequality into the Armijo condition, noting that $\nabla f\left(\mathbf{x}_k\right)^\top \mathbf{p}_k < 0$,

$$f\left(\mathbf{x}_{k+1}\right) \leq f\left(\mathbf{x}_k\right) - \frac{c_1 \left(1 - c_2\right)}{\gamma} \frac{\left(\nabla f\left(\mathbf{x}_k\right)^\top \mathbf{p}_k\right)^2}{\left\| \mathbf{p}_k \right\|^2}$$

Letting $c = c_1 (1 - c_2) / \gamma$ and $\delta_k = \cos \theta_k$, we can rewrite this as

$$f(\mathbf{x}_{k+1}) \leq f(\mathbf{x}_k) - c\delta_k^2 \|\nabla f(\mathbf{x}_k)\|^2 .$$

Summing over all $j \leq k$,

$$f(\mathbf{x}_{k+1}) \leq f(\mathbf{x}_0) - c \sum_{j=0}^{k} \delta_j^2 \|\nabla f(\mathbf{x}_j)\|^2$$

Since f is bounded below, there exists some $M > 0$ such that

$$f\left(\mathbf{x}_0\right) - f\left(\mathbf{x}_{k+1}\right) \leq M$$

for all $k \in \mathbb{N}$, so

$$\sum_{j=0}^{k} \delta_j^2 \left\|\nabla f\left(\mathbf{x}_j\right)\right\|^2 \leq \frac{M}{c}$$

Since each term in the sum is nonnegative and the sequence of partial sums is bounded above,

$$\sum_{j=0}^{\infty} \delta_j^2 \left\| \nabla f\left(\mathbf{x}_j\right) \right\|^2$$

is convergent. It follows that

$$\lim_{k \to \infty} \delta_k^2 \left\| \nabla f\left(\mathbf{x}_k\right) \right\|^2 = 0$$

Since $\delta_k \geq \delta > 0$

$$0 \leq \delta^2 \left\|\nabla f\left(\mathbf{x}_k\right)\right\|^2 \leq \delta_k^2 \left\|\nabla f\left(\mathbf{x}_k\right)\right\|^2 \to 0$$

Hence $\left\|\nabla f\left(\mathbf{x}_k\right)\right\| \to 0$.

The conditions in Theorem 1.1 are quite natural in optimization problems: minimization algorithms are performed only on functions bounded below, and the Lipschitz condition in the derivative is a standard assumption.

A quick word on the importance of δ_k. Note that θ_k is the angle measure between $\nabla f(\mathbf{x}_k)$ and $\mathbf{p}_k$. In the steepest descent algorithm we choose $\mathbf{p}_k = -\nabla f(\mathbf{x}_k)$ so that $\delta_k^2 = 1$ for all k. In particular, the sequence of gradients converges. In Newton's method applied to a strictly convex function f,

$$\delta_k = \frac{1}{\kappa\left(\nabla^2 f(\mathbf{x}_k)\right)},$$

where

$$\kappa(X) = \frac{\lambda_{\max}(X)}{\lambda_{\min}(X)}$$

is the condition number of a matrix $X \in \mathbb{S}^n_{++}$. If the condition numbers of the Hessians are uniformly bounded above, the angle condition is satisfied.

Aside: Convergence rates of steepest descent and Newton's method

These are not required but we mention the convergence rates for cultural reasons. These results come in many forms but basically they all state that steepest descent converges linearly and Newton method converges quadratically.

Theorem (Convergence rate of steepest descent)

If

1. $f \in C^2(\mathbb{R}^n)$,
2. $\mathbf{x}_k$ *generated by steepest descent with exact line search converges to* $\mathbf{x}_*$,
3. $\nabla f(\mathbf{x}_*) = \mathbf{0}$ *and* $\nabla^2 f(\mathbf{x}_*) \succ 0$, *then for any* r *such that*

$$\frac{\lambda_{\max} - \lambda_{\min}}{\lambda_{\max} + \lambda_{\min}} < r < 1,$$

and k *sufficiently large,*

$$f(\mathbf{x}_{k+1}) - f(\mathbf{x}_*) \leq r^2 \left(f(\mathbf{x}_k) - f(\mathbf{x}_*)\right).$$

$\lambda_{\max}$ *and* $\lambda_{\min}$ *denote the largest and smallest eigenvalues of* $\nabla^2 f(\mathbf{x}_*)$.

The proof is beyond the scope of the course and in lieu of it we will examine the convergence rate in the special case when f is quadratic.

i.e., $f : \mathbb{R}^n \to \mathbb{R}$ given by $f(\mathbf{x}) = \frac{1}{2}\mathbf{x}^\top A\mathbf{x} - \mathbf{b}^\top \mathbf{x}$, where $A \in \mathbb{S}^n_{++}$ and $\mathbf{b} \in \mathbb{R}^n$.

Note that

- f is convex since $\nabla^2 f(\mathbf{x}) = A \succ 0$
- $\nabla f(\mathbf{x}) = A\mathbf{x} - \mathbf{b}$
- $\mathbf{x}_* = A^{-1}\mathbf{b}$.

The exact line search sets $\alpha_k = \alpha$, where α is the minimizer of

$$g(\alpha) = f\left(\mathbf{x}_k - \alpha \nabla f\left(\mathbf{x}_k\right)\right) = \frac{1}{2}\left(\mathbf{x}_k - \alpha \mathbf{p}_k\right)^\top A\left(\mathbf{x}_k - \alpha \mathbf{p}_k\right) - \mathbf{b}^\top\left(\mathbf{x}_k - \alpha \mathbf{p}_k\right).$$

By Homework 6, Problem 1(c), we have

$$\alpha_k = \operatorname{argmin}_{\alpha \in \mathbb{R}} g(\alpha) = \frac{\mathbf{p}_k^\top \mathbf{p}_k}{\mathbf{p}_k^\top A \mathbf{p}_k}$$

Hence the iterations for steepest descent with exact line search are given by

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \left[\frac{\nabla f(\mathbf{x}_k)^\top \nabla f(\mathbf{x}_k)}{\nabla f(\mathbf{x}_k)^\top A \nabla f(\mathbf{x}_k)} \right] \nabla f(\mathbf{x}_k).$$

Homework 6, Problem 3 contains a special case of this computation. Since $\nabla f(\mathbf{x}_k) = A(\mathbf{x}_k - \mathbf{x}_*)$,

$$\begin{aligned}
\|\mathbf{x}_{k+1} - \mathbf{x}_*\|_A^2 &= \left[1 - \frac{\left(\mathbf{p}_k^\top \mathbf{p}_k\right)^2}{\left(\mathbf{p}_k^\top A \mathbf{p}_k\right)\left(\mathbf{p}_k^\top A^{-1} \mathbf{p}_k\right)}\right] \|\mathbf{x}_k - \mathbf{x}_*\|_A^2 \\
&= \left[1 - \frac{\|\mathbf{p}_k\|^2}{\|\mathbf{p}_k\|_A^2 \|\mathbf{p}_k\|_{A^{-1}}^2}\right] \|\mathbf{x}_k - \mathbf{x}_*\|_A^2
\end{aligned}$$

Recall that $\|\mathbf{x}\|_A := \sqrt{\mathbf{x}^\top A\mathbf{x}}$ is the Mahalanobis norm of $\mathbf{x}$ with weight matrix A. This bound can be further bounded by $(\lambda_{\max} - \lambda_{\min}) / (\lambda_{\max} + \lambda_{\min})$ as in theorem above. This requires the Kantorovich inequality and we won't discuss it to keep to our promise that we only need Cauchy-Schwatz and triangle inequalities in this course. In fact,

$$\frac{\lambda_{\max} - \lambda_{\min}}{\lambda_{\max} + \lambda_{\min}} = \frac{\kappa(A) - 1}{\kappa(A) + 1}$$

where $\kappa(A)$ is the condition number of the matrix A.

Convergence of Newton's method

Sufficient conditions for quadratic convergence of Newton's method

Theorem (Convergence rate of Newton method)

If

1. $f \in C^2(\mathbb{R}^n)$,
2. $\nabla^2 f \in \mathrm{Lip}_\gamma(\mathbb{R}^n)$,
3. $\mathbf{x}_0$ *is sufficiently close to* $\mathbf{x}_*$,
4. $\mathbf{x}_*$ *is a strict local minimizer, then Newton's method with step size* $\alpha_k = 1$ *for all* $k = 0, 1, \ldots$ *converges to* $\mathbf{x}_*$ *where* $\nabla f(\mathbf{x}_*) = \mathbf{0}$ *and* $\nabla^2 f(\mathbf{x}_*) \succ 0$. *Furthermore rate of convergence of both* $\lim_{k\to\infty} \mathbf{x}_k = \mathbf{x}_*$ *and* $\lim_{k\to\infty} \nabla f(\mathbf{x}_k) = \mathbf{0}$ *is quadratic.*

Quasi-Newton methods

Quasi-Newton methods

We've looked closely at two optimization methods: steepest descent and Newton's method.

Steepest descent is easy to compute at each iteration – we simply use the search direction $-\nabla f(\mathbf{x}_k)$. However, it converges linearly.

By contrast, Newton's method converges quadratically. But here the seach direction at every step is $\mathbf{p}_k = -\nabla^2 f(\mathbf{x}_k)^{-1} \nabla f(\mathbf{x}_k)$. This requires evaluation of a Hessian (n^2 evaluations) and matrix inversion at each iteration, both of which are costly.

Quasi-Newton methods are an "in-between" alternative to Newton's method, requiring fewer function evaluations and supplying superlinear convergence.

A side note: intuition behind the multivariate Newton's method

Because I didn't describe this very well in the previous lecture, let's go through it again.

In Newton's method for optimization, we want to move in the direction that brings the gradient to zero.

How much do we want to move in each direction? Well, that depends on how big the gradient currently is in that direction, as well as how quickly gradients in that direction change.

Suppose our function is $f(x, y) = x^2 + y^2 + xy$. Then the gradient is $\nabla f(x, y) = [2x + y, 2y + x]$ and the Hessian is $\nabla^2 f(x, y) = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$.

Let's make some plots.

```
Unable to display output for mime type(s): text/html
```

```
Unable to display output for mime type(s): text/html
```

```
Unable to display output for mime type(s): text/html
```

```
Unable to display output for mime type(s): text/html
```

```
Unable to display output for mime type(s): text/html
```

What Newton's method does is say, "suppose our step size is 1 and that our Hessian is constant. Which direction should we move in to bring our gradient to zero, in just one step?"

The expected change in the gradient is given by the Hessian times the step direction. So we want to solve

$$\nabla^2 f(\mathbf{x}_k)\,\mathbf{p}_k = -\nabla f(\mathbf{x}_k)\,.$$

Then we just need to solve for $\mathbf{p}_k$ in the above equation:

$$\mathbf{p}_k = -\nabla^2 f(\mathbf{x}_k)^{-1}\,\nabla f(\mathbf{x}_k)\,.$$

Back to quasi-Newton methods

The basic idea: use an approximation to the Hessian.

Specifically, at each iteration we will choose an approximation H_k to the Hessian $\nabla^2 f(\mathbf{x}_k)$.

We will use that approximation to get a search direction $\mathbf{p}_k = -H_k^{-1} \nabla f(\mathbf{x}_k)$.

What will make a good approximation? Well, we could say that we want H_k to get close to the actual Hessian, i.e.

$$\left\| H_k - \nabla^2 f\left(\mathbf{x}_k\right) \right\| \to 0.$$

However, this is stronger than we actually need. Since we are only using H_k to get a search direction, we only need the search directions it predicts to be close to the search direction of Newton's method, i.e.

$$\left\| H_k^{-1} \nabla f\left(\mathbf{x}_k\right) - \nabla^2 f\left(\mathbf{x}_k\right)^{-1} \nabla f\left(\mathbf{x}_k\right) \right\| \to 0.$$

Notice that this leaves a lot of freedom in choosing H_k. We only need that H_k^{-1} **times our current gradient** is close to the Newton direction. But there may be many matrices that do this.

OK, so how do we choose H_k out of the many possible matrices which satisfy this equation?

$$\left\| H_k^{-1} \nabla f\left(\mathbf{x}_k\right) - \nabla^2 f\left(\mathbf{x}_k\right)^{-1} \nabla f\left(\mathbf{x}_k\right) \right\| \to 0$$

The secant equation

How does the gradient change from one iteration to the next?

From the Taylor expansion, we know that

$$\nabla f\left(\mathbf{x}_k\right) \approx \nabla f\left(\mathbf{x}_{k-1}\right) + \nabla^2 f\left(\mathbf{x}_k\right)\left(\mathbf{x}_k - \mathbf{x}_{k-1}\right).$$

We can rearrange this to get

$$\nabla f\left(\mathbf{x}_k\right) - \nabla f\left(\mathbf{x}_{k-1}\right) \approx \nabla^2 f\left(\mathbf{x}_k\right)\left[\mathbf{x}_k - \mathbf{x}_{k-1}\right].$$

This suggests that could choose H_k to be a matrix that exactly satisfies this equation:

$$\nabla f\left(\mathbf{x}_k\right) - \nabla f\left(\mathbf{x}_{k-1}\right) = H_k(\mathbf{x}_k - \mathbf{x}_{k-1}).$$

$$\nabla f\left(\mathbf{x}_k\right) - \nabla f\left(\mathbf{x}_{k-1}\right) = H_k(\mathbf{x}_k - \mathbf{x}_{k-1}).$$

We can simplify this a bit by defining $\mathbf{s}_k = \mathbf{x}_k - \mathbf{x}_{k-1}$ and $\mathbf{y}_k = \nabla f\left(\mathbf{x}_k\right) - \nabla f\left(\mathbf{x}_{k-1}\right)$. Then the equation becomes

$$\mathbf{y}_k = H_k \mathbf{s}_k.$$

This is called the **secant equation**.

Intuitively, if we pick an H_k that satisfies the secant equation, then we know that it will approximate the Hessian.

Or rather, to be more precise,it will predict a change in the gradient **as we move in the step direction** that is similar to the change in the gradient, in the same direction, that the Hessian predicts.

$$H_k \mathbf{s}_k \approx \nabla^2 f\left(\mathbf{x}_k\right) \mathbf{s}_k.$$

Davidon-Fletcher-Powell

In this method, we'll suppose we have already computed a current approximation H_k.

We are going to update this to get a new approximation H_{k+1}, using the following criteria:

1. H_{k+1} should satisfy the secant equation
2. Given the first criterion, H_{k+1} should be as close as possible to H_k

Remember, the Hessian allows us to predict how the gradient will change as we move in any direction.

When we take a step in a direction $\mathbf{s}_k$, we estimate the curvature of the function in that direction by $\mathbf{y}_k/\mathbf{s}_k$. If we take a second step in a different direction, we use the curvature in **that** direction to update our estimate of the Hessian.

Geometrically, each update ensures that the approximation matches the curvature of the objective function along the most recent step direction.

As we take more steps, the approximation captures curvature information in more directions, gradually building a more complete picture of the function's behavior.

Formally, we set H_{k+1} to be the minimizer of the following problem:

$$\begin{aligned} &\text{minimize} \, \|H - H_k\| \\ &\text{subject to } H = H^\top, \, \underbrace{H\mathbf{s}_k = \mathbf{y}_k}_{\text{secant equation}} \end{aligned} \tag{$\star$}$$

where $s_k = \mathbf{x}_{k+1} - \mathbf{x}_k$ and $\mathbf{y}_k = \nabla f\left(\mathbf{x}_{k+1}\right) - \nabla f\left(\mathbf{x}_k\right)$.

Why can't we just solve the secant equation directly?

Intuitively, the solution to the secant equation can be thought of as

$$H_k = \frac{\nabla f\left(\mathbf{x}_{k+1}\right) - \nabla f\left(\mathbf{x}_k\right)}{\mathbf{x}_{k+1} - \mathbf{x}_k} \approx \nabla^2 f\left(\mathbf{x}_{k+1}\right).$$

This, of course, is not allowed since the numerator and denominator are vectors.

Ideally, we would like

1. H_{k+1} to be symmetric
2. H_{k+1} to be positive definite
3. H_{k+1} to satisfy the secant equation.

The solution to $(\star)$ is given by the Davidon-Fletcher-Powell (DFP) update:

$$H_{k+1} = \left(I - \rho_k \mathbf{y}_k \mathbf{s}_k^\top\right) H_k \left(I - \rho_k \mathbf{s}_k \mathbf{y}_k^\top\right) + \rho_k \mathbf{y}_k \mathbf{y}_k^\top,$$

where $\rho_k = 1/\left(\mathbf{y}_k^\top \mathbf{s}_k\right)$.

The DFP updates for the inverse Hessian $B_k := H_k^{-1}$ can be computed efficiently using the Sherman-Woodbury-Morisson formula (see Appendix):

$$B_{k+1} = B_k - \frac{B_k \mathbf{y}_k \mathbf{y}_k^\top B_k}{\mathbf{y}_k^\top B_k \mathbf{y}_k} + \frac{\mathbf{s}_k \mathbf{s}_k^\top}{\mathbf{y}_k^\top \mathbf{s}_k}.$$

Note that positive definiteness of H_k (resp. B_k) implies the positive definiteness of H_{k+1} (resp. B_{k+1}) is positive definite.

In particular, if we choose H_0 or B_0 to be positive definite, all subsequent H_k and B_k will be positive definite.

The most common choices are $B_0 = I$ or $H_0 = I$, though one can also choose $H_0 = \nabla^2 f(\mathbf{x}_0)$ (provided the Hessian is positive definite) or $H_0 = \operatorname{diag}(\nabla^2 f(\mathbf{x}_0))$, with sign changes to ensure positive definiteness.

Broyden-Fletcher-Goldfarb-Shanno

Recall that our goal is to approximate the inverse Hessian $B_k \approx \nabla^2 f(\mathbf{x}_k)^{-1}$.

Instead of approximating the Hessian and inverting, we could have started the other way around by finding a formula for the approximate inverse Hessian.

If B_k has been computed, we set B_{k+1} to be the minimizer of the following problem:

$$\begin{aligned} &\text{minimize} \|B - B_k\| \\ &\text{subject to } B = B^\top, \quad B\mathbf{y}_k = \mathbf{s}_k \end{aligned} \tag{**}$$

Note that this is the dual of $(\star)$: the equation $B\mathbf{y}_k = \mathbf{s}_k$ is solved by the inverse to the solution to the secant equation $H\mathbf{s}_k = \mathbf{y}_k$.

The solution to ($**$ is given by the **Broyden-Fletcher-Goldfarb-Shanno (BFGS) update**:

$$B_{k+1} = \left(I - \rho_k \mathbf{s}_k \mathbf{y}_k^\top\right) B_k \left(I - \rho_k \mathbf{y}_k \mathbf{s}_k^\top\right) + \rho_k \mathbf{s}_k \mathbf{s}_k^\top$$

where $\rho_k = 1/\left(\mathbf{y}_k^\top \mathbf{s}_k\right)$.

If desired, the approximate Hessian $H_k := B_k^{-1}$ can be computed with Sherman-Morisson:

$$H_{k+1} = H_k - \frac{H_k \mathbf{s}_k \mathbf{s}_k^\top H_k}{\mathbf{s}_k^\top H_k \mathbf{s}_k} + \frac{\mathbf{y}_k \mathbf{y}_k^\top}{\mathbf{y}_k^\top \mathbf{s}_k}.$$

Both BFGS and DFP are special cases of **Broyden class updates**:

$$H_{k+1} = (1 - \varphi_k)\, H_{k+1}^{\mathrm{BFGS}} + \varphi_k H_{k+1}^{\mathrm{DFP}}.$$

Alternative way to obtain DFP and BFGS

Consider the constrained minimization problem (next lecture) over $\mathbb{S}^n_{++}$:

$$\begin{aligned} &\text{minimize} \quad \operatorname{tr}\left(H_k^{-1}X\right) - \log\det\left(H_k^{-1}X\right) \\ &\text{subject to } X\mathbf{s}_k = \mathbf{y}_k. \end{aligned}$$

Minimizer is $X = H_{k+1}$ where

$$H_{k+1} = H_k - \frac{H_k\mathbf{s}_k\mathbf{s}_k^\top H_k}{\mathbf{s}_k^\top H_k\mathbf{s}_k} + \frac{\mathbf{y}_k\mathbf{y}_k^\top}{\mathbf{y}_k^\top\mathbf{s}_k},$$

```
Algorithm 6 Quasi–Newton using DFP updates
Require: starting point \(\mathbf{x}_{0} \in \Omega\), tolerance \(\va
    procedure \(\operatorname{DFP}\left(x_{0}, \varepsilon, B_{0}\righ
        \(k \leftarrow 0\)
        while \(\left\|\nabla f\left(\mathbf{x}_{k}\right)\right\|>\va
            Quasi–Newton step: \(\mathbf{p}_{k} \leftarrow-B_{k} \nabl
            Line search: call backtracking line search to find suitabl
            Update: \(\mathbf{x}_{k+1} \leftarrow \mathbf{x}_{k}+\alpha
            Update inverse Hessian:
\[
\begin{aligned}
\mathbf{s}_{k} & \leftarrow \mathbf{x}_{k+1}-\mathbf{x}_{k} \\
\mathbf{y}_{k} & \leftarrow \nabla f\left(\mathbf{x}_{k+1}\right)-\nab
B_{k+1} & \leftarrow B_{k}-\frac{B_{k} \mathbf{y}_{k} \mathbf{y}_{k}^{
\end{aligned}
\]
            \(k \leftarrow k+1\)
        end while
        return \(\mathrm{x}_{*}:=\mathrm{x}_{k}\)
    end procedure
```

i.e., BFGS update.

Now consider over $\mathbb{S}^n_{++}$:

$$\begin{aligned} \text{minimize} \quad & \operatorname{tr}\left(H_k X^{-1}\right) - \log\det\left(H_k X^{-1}\right) \\ \text{subject to} \quad & X\mathbf{s}_k = \mathbf{y}_k. \end{aligned}$$

Minimizer is $X = H_{k+1}$ where

$$H_{k+1} = \left(I - \frac{\mathbf{y}_k \mathbf{s}_k^\top}{\mathbf{s}_k^\top \mathbf{y}_k}\right) H_k \left(I - \frac{\mathbf{s}_k \mathbf{y}_k^\top}{\mathbf{s}_k^\top \mathbf{y}_k}\right) + \frac{\mathbf{y}_k \mathbf{y}_k^\top}{\mathbf{s}_k^\top \mathbf{y}_k},$$

i.e., DFP update.

Note that quasi-Newton methods are so-called two-step methods, unlike steepest descent and Newton, which only involves the gradient (or Hessian) at step k, they require that we use the gradients at two successive steps since we need both to compute

$$\mathbf{y}_k = \nabla f\left(\mathbf{x}_{k+1}\right) - \nabla f\left(\mathbf{x}_k\right).$$

Barzilai-Borwein step size

The key idea behind the Barzilai-Borwein step size described earlier is the same one that inspires quasi-Newton methods. Here we assume that the approximate Hessian B_k takes the form $B_k = (1/\alpha_k)\, I$ for some scalar $\alpha_k > 0$. So we solve

$$\min_{\alpha>0} \left\| \frac{1}{\alpha} \mathbf{y}_k - \mathbf{s}_k \right\|^2$$

which gives us

$$\alpha_{k+1} = \frac{\mathbf{y}_k^\top \mathbf{s}_k}{\mathbf{y}_k^\top \mathbf{y}_k} = \frac{\left| \left(\nabla f\left(\mathbf{x}_{k+1}\right) - \nabla f\left(\mathbf{x}_k\right) \right)^\top \left(\mathbf{x}_{k+1} - \mathbf{x}_k \right) \right|}{\left\| \nabla f\left(\mathbf{x}_{k+1}\right) - \nabla f\left(\mathbf{x}_k\right) \right\|^2},$$

the Barzilai-Borwein step size defined earlier.

Like quasi-Newton method, this is a two-step method.

Allowing an additional step allows us to circumvent the methods that we discussed in the previous lecture for obtaining step sizes (backtracking line search and exact line search).

The Barzilai-Borwein step size has become one of the most popular choices for step size, aka learning rate, especially when used in conjunction with various variants of stochastic gradient descent.